

Cash-out Forecast Console

The Complete Manual — every idea, every model, every line of the stack, from scratch.

Predictive analytics

GIS forecasting

Leakage-safe ML

Predictive Analytics for Cybercrime Cash-out Forecasting

A leakage-safe, honesty-first prototype for the Indian Cyber Crime Coordination Centre.

Contents

FRONT MATTER

How to read this manual

PART I · ORIENTATION

- 1 · What this project is
- 2 · The problem, in real terms
- 3 · The big idea
- 4 · Glossary of every term

PART II · THE SYSTEM AT A GLANCE

- 5 · The five-step pipeline
- 6 · The four I4C deliverables
- 7 · Architecture & data flow
- 8 · The complete technology stack
- 9 · The repository, file by file

PART III · THE DATA FOUNDATION

- 10 · Why the data is synthetic
- 11 · The synthetic world
- 12 · The generator dials
- 13 · ATMs, geography & H3 cells
- 14 · The sealed corpus
- 15 · The data contracts

PART IV · TRACING THE MONEY

- 16 · The as-of principle
- 17 · Money-flow reconstruction
- 18 · Graph features & the mule score
- 19 · Candidate ATM generation

PART V · THE FORECASTING ENGINE

- 20 · The as-of feature store
- 21 · Every feature, grouped
- 22 · M1 — the hazard curve
- 23 · M2 — the channel
- 24 · M3 — location & abstention
- 25 · Training & inference

PART VI · HONESTY & EVALUATION

26 · The six non-negotiable rules

27 · The leakage test suite

28 · Evaluation: splits & baselines

29 · The numbers & how to defend them

PART VII · DECISION & ALERTS

30 · The plain-rules decision layer

31 · Alert routing & dedup

32 · The API, endpoint by endpoint

PART VIII · THE CONSOLE

33 · Design philosophy

34 · The three views

35 · The live money-flow map

36 · The cinematic replay

37 · The SOAR ticket lifecycle

PART IX · RUNNING & OPERATING

38 · Setup from zero

39 · The live demo script

40 · Troubleshooting

PART X · RESEARCH & ROADMAP

41 · The GNN research track

42 · Known limitations

43 · Roadmap to production

Appendix · Command & file reference

How to read this manual

FRONT MATTER

This is the single book that explains the entire Cash-out Forecast Console — from the human problem it solves, down to the last library in the stack and the last panel on the screen. It assumes **no prior knowledge**. If you have never seen the code, start at page one and keep going; by the end you will be able to run it, extend it, and defend every design decision to a technical judge.

The manual is written in ten parts. **Parts I–II** give you the whole idea and a map of the system. **Parts III–V** descend into the three technical pillars: the data, the money-tracing graph, and the machine-learning engine. **Part VI** is the most important part for credibility — it explains the six rules that stop the system from cheating, and how each rule is mechanically enforced. **Parts VII–VIII** cover the officer-facing product: the decision rules, the alert system, and the command console. **Parts IX–X** tell you how to run it, demo it, and where it goes next.

READING PATHS

The presenter / judge-facing team: Parts I, VI, and the demo script (Ch. 39) are enough to stand up and explain the project convincingly.

The engineer joining the team: read straight through; every module is mapped to its file.

The person in a hurry: a separate 15-page *Field Guide* distils this whole book into the essentials.

Throughout, colour-coded boxes flag three things: **red** boxes are the non-negotiable rules that keep the science honest; **amber** boxes are cautions and things people get wrong; **green** boxes are the key takeaways worth memorising.

What this project is

PART I · ORIENTATION

When someone is scammed online, the fraudster does not keep the money. Within minutes they move it through a chain of rented "mule" bank accounts and then pull it out as cash at an ATM. Once it is cash, it is effectively unrecoverable. This system takes a fraud complaint, reconstructs where the money went, and then forecasts — for each account still holding the stolen money — **whether it will be cashed out, when, through what channel, and roughly at which ATM**. The purpose is to freeze the account before the cash is withdrawn, and, if that is impossible, to tell police where to look.

The one-sentence pitch

"We don't just detect fraud after the fact — we forecast the cash-out, so police can freeze the account in time, and when they can't, we tell them which area to watch."

That sentence contains the entire thesis. Every design decision in the codebase exists to make that forecast *accurate, fast enough to act on, and honest about its own uncertainty*. Those three properties — accuracy, timeliness, and honesty — are the axes on which the project should be judged, and they are the reason the architecture looks the way it does.

Where it fits

The prototype is built against the workflow of the **Indian Cyber Crime Coordination Centre (I4C)** under the Ministry of Home Affairs, and the national complaint system (the citizen files a report; a fourteen-digit acknowledgement number, the `ack_no`, identifies the case). The console is designed to sit on an investigating officer's desk: a queue of live complaints on the left, a forecast and a live money-flow map in the centre, and a set of concrete actions — *freeze this account, dispatch to this station, keep tracing* — on the right.

WHAT "CASH-OUT FORECASTING" MEANS PRECISELY

It is *not* transaction fraud scoring (is this payment fraudulent?). The fraud has already happened. It is a **forward-looking, time-boxed prediction** about the fate of already-stolen money: given the money is now sitting in account X, what is the probability it becomes untraceable cash in the next 10, 20, 30, 45, or 60 minutes, and where.

2 · The problem, in real terms

PART I · ORIENTATION

To understand why the system is shaped the way it is, you have to understand the timeline of a real cash-out, because the whole design is a race against that clock.

The anatomy of a cash-out

1. **The fraud.** A victim is tricked into sending money — a fake KYC update, a "your account is blocked" call, a bogus refund. One transaction leaves their account. This is the *seed transaction*.
2. **Layering.** The money does not sit still. Within seconds to minutes it is split and forwarded through several mule accounts. Splitting it makes it harder to trace and spreads the withdrawal across many people and ATMs. This is the classic money-laundering *layering* step, compressed into minutes.
3. **Cash-out.** Each mule withdraws their share as cash at a local ATM (or a POS "cash-at-till"). The instant it is cash, the paper trail ends.

The brutal arithmetic of time

The reason this is hard — and the reason a *forecast* is valuable rather than a report — is the mismatch between two delays:

- The **forwarding delay** (fraud → cash) is typically minutes.
- The **reporting delay** (fraud → complaint filed) is typically around **110 minutes**.

THE HONEST, UNCOMFORTABLE FACT

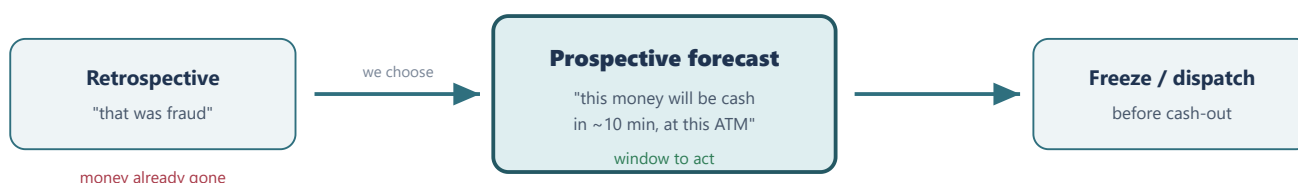
In roughly **two out of three complaints, the money was already cashed out before the complaint was even filed**. A prototype that pretends otherwise is lying. This system is built to be useful in the cases where action is still possible, and — just as importantly — to make the case for the two structural fixes that would widen that window: *faster reporting* (auto-alerts at the bank) and *proactive freezing*.

This single fact reframes the goal. The winning move is not "catch them at the ATM" — by the time you know, the cash is gone. The winning move is **"freeze the account before the ATM."** Freezing is a database action that takes seconds; a police dispatch takes tens of minutes. So the system optimises for the thing that can actually beat the clock: a fast, well-justified freeze recommendation, routed to the right desk, with a geographic fallback for when freezing fails.

3 · The big idea

PART I · ORIENTATION

Most fraud tooling is *retrospective*: it labels transactions as fraudulent so they can be investigated later. That is necessary but it does not save the money. This project makes a deliberate bet on being *prospective*: it treats the stolen money as a moving object and forecasts its next state.



The project's central bet: turn fraud tooling from a report into a forecast with a window to act.

The consequence of that bet runs all the way down. Because we forecast the future, every feature must be computed from the past only (Part IV/VI). Because officers must act on the forecast, the output is not a probability in isolation but a *tiered recommendation with a reason* (Part VII). And because a false "go to ATM #14" wastes a patrol, the location model is allowed to *abstain* — to say "I only know the neighbourhood" — rather than bluff (Chapter 24). Honesty is not decoration here; it is a functional requirement.

4 · Glossary of every term

PART I · ORIENTATION

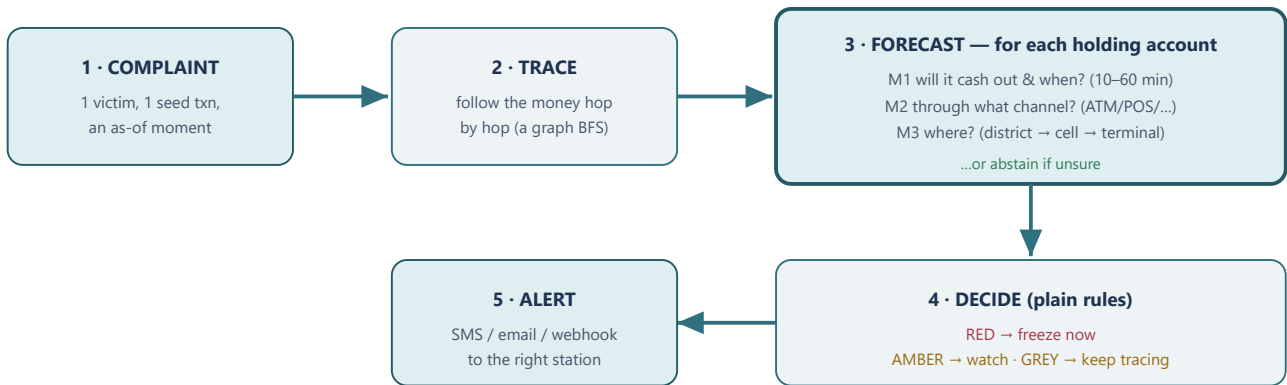
Every domain term used in this manual and in the code, defined once. Keep a finger here on your first read.

Term	Meaning
<code>complaint / ack_no</code>	A filed cybercrime report. <code>ack_no</code> is its 14-digit acknowledgement number and the primary key of a case.
<code>seed transaction</code>	The one disputed payment the victim reported. The root of the money trail.
<code>victim</code>	The account the seed transaction left. Blue node on the map.
<code>mule account</code>	A (usually rented or bought) bank account used to receive and move stolen money.
<code>ring</code>	A small set of mule accounts operated together (default size 3). They share ATMs and behaviour — a key signal.
<code>layering</code>	Splitting and forwarding stolen money through several accounts to obscure the trail.
<code>trail</code>	The reconstructed chain of accounts and edges the stolen money passed through, up to <code>as_of</code> .
<code>taint / tainted amount</code>	The portion of an account's balance that is attributable to the stolen money, tracked proportionally through each split.
<code>leaf / holding account</code>	An account that has received stolen money but not yet forwarded/withdrawn it — a live freeze target.
<code>cash-out</code>	The moment money becomes physical cash (ATM withdrawal or POS cash-at-till). The end of the trail.
<code>channel</code>	How money leaves an account: ATM, POS, onward transfer, or dormant.
<code>as-of (as_of)</code>	The exact moment a prediction is made. Features may read only data with <code>ts < as_of</code> . The core anti-leakage discipline.
<code>hazard curve</code>	The rising cumulative probability of cash-out over the next 10–60 minutes (model M1).
<code>window</code>	The steepest-rise interval read off the hazard curve — the "act by" time band.
<code>cell</code>	An H3 hexagon at resolution 7 (~5 km ²), a neighbourhood-sized area. Res-8 (~0.5 km ²) is the fine tier.
<code>terminal</code>	A single ATM (the finest location the system may name).
<code>abstention</code>	The location model refusing to name one ATM when the distribution is too flat, giving the cell or district instead.
<code>KYC centroid</code>	The coordinates of an account's registered address PIN code — the only "location" allowed as a feature.
<code>golden hour</code>	The first 60 minutes after the fraud — the best window to freeze.
<code>lead time</code>	Minutes between raising the alarm and the actual cash-out. Positive = we were early.
<code>tier (RED/AMBER/GREY)</code>	The urgency of a case: RED = freeze now; AMBER = watch/brief; GREY = keep tracing.
<code>sealed corpus</code>	A deliberately different test dataset the model never trained on — the proof it generalises.
<code>SHO</code>	Station House Officer — the routing target for a dispatched alert.

5 · The five-step pipeline

PART II · THE SYSTEM AT A GLANCE

The whole system is a pipeline of five stages. A complaint comes in; the money is traced; each holding account is forecast; plain rules turn forecasts into decisions; and alerts go out. Everything else in this manual is a detailed view of one of these five boxes.



The five-step pipeline. Stages 3 and 4 are the intellectual core; stages 1, 2 and 5 make it operational.

Two properties of this pipeline are worth stating up front. First, it is **rebuilt fresh for every prediction** — there is no giant precomputed graph of the whole world sitting around, because that would be the first and worst way to leak future information. Second, the intelligence is deliberately concentrated in stages 3 and 4 and kept *out* of the plumbing: the decision layer (stage 4) is plain, readable rules, not a black box, so an officer can always read *why* an alert fired.

6 · The four I4C deliverables

PART II · THE SYSTEM AT A GLANCE

The problem statement asks for four things. The prototype delivers all four, and it is useful to know exactly which part of the code is which deliverable.

#	Deliverable	Where it lives
1	Predictive analytics engine — the AI that forecasts will / when / how / where	<code>ml/</code> (M1, M2, M3) built on <code>graphx/</code>
2	GIS risk forecasting & heat-map with filtering	<code>GET /hotspots</code> + the live Leaflet map in <code>web/index.html</code>
3	Law-enforcement interface — triage queue, per-case intelligence, freeze/dispatch, case management	<code>api/main.py</code> + the console (Command Centre, Case view, Case Centre)
4	Alert & notification system (SMS / email / API) with a delivery log	<code>POST /alerts/dispatch</code> , <code>GET /alerts</code>

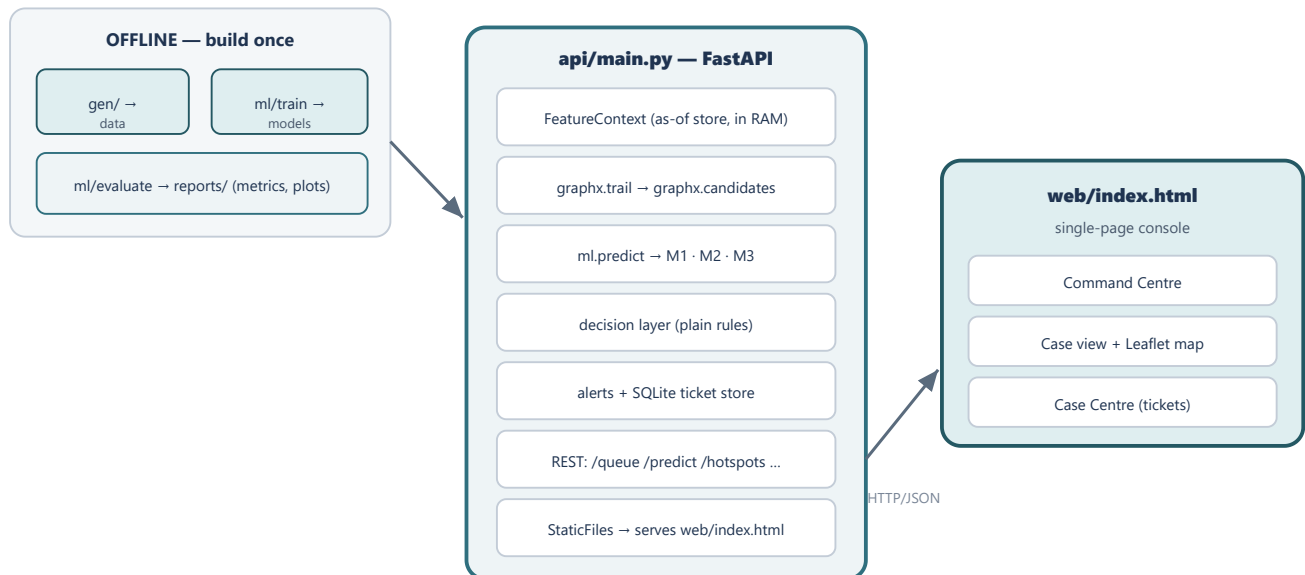
WHY THIS MAPPING MATTERS IN A DEMO

When a judge asks "show me deliverable 3," you can point to a specific screen and a specific file. Nothing in this project is aspirational — every required deliverable is a running piece of software you can open.

7 · Architecture & data flow

PART II · THE SYSTEM AT A GLANCE

The system is a single Python backend that serves a single-file web console. There is no database server, no message queue, no cloud dependency — it runs entirely on one machine from a folder of Parquet files and trained model artefacts. That simplicity is a feature: the whole thing is reproducible and auditable.



One backend process, built from offline artefacts, serving one browser console over REST/JSON.

The **offline** half (left) runs a handful of times: it generates the synthetic world, trains the three models, and produces the evaluation report. Its outputs are just files — Parquet tables under `data/` and joblib model files under `models/`. The **online** half (centre) is the FastAPI server: on startup it loads everything into memory as a `FeatureContext`, and thereafter every request is answered by re-tracing the trail and re-running the models *as of* the requested moment. The **console** (right) is a single HTML file served as a static asset; it talks to the backend purely over JSON.

8 · The complete technology stack

PART II · THE SYSTEM AT A GLANCE

Every library the project depends on, what it does here, and why it was chosen. Nothing exotic — the stack is deliberately boring and reproducible.

Core data & modelling (the `ml/`, `gen/`, `graphx/` side)

Library	Role in this project
Python 3.12	The single implementation language for the whole backend and pipeline.
pandas	All tabular data: accounts, transactions, complaints, ATMs; the as-of feature store is built on sorted pandas structures.
numpy	Numeric arrays, the fast <code>searchsorted</code> as-of cuts, softmax, entropy, haversine.
scikit-learn	Model wrappers, calibration, metrics (ROC-AUC), the logistic mule-score, train/test splitting.
xgboost	The gradient-boosted trees behind M1 (hazard) and M3 (location ranker) — strong on tabular, mixed-scale features.
networkx	Graph utilities for the trail and ring structure.
h3	Uber's hexagonal spatial index (v4 API). Res-7 $\approx 5 \text{ km}^2$ operational cells; res-8 $\approx 0.5 \text{ km}^2$ fine tier. The backbone of the location hierarchy.
pyarrow	Reads/writes the Parquet data files — compact, typed, fast columnar storage.
joblib	Serialises trained models to <code>models/*.joblib</code> .
matplotlib	The evaluation plots (reliability, coverage/accuracy, lead-time) in <code>reports/</code> .

Service & contract layer (the `api/` side)

FastAPI	The REST backend: routing, validation, automatic OpenAPI docs at <code>/docs</code> .
uvicorn	The ASGI server that runs FastAPI.
pydantic	The frozen data contracts in <code>contracts.py</code> — every shape crossing a boundary is a validated model.
SQLite (stdlib)	The Case Centre ticket store — WAL mode, seeded once, for the SOAR-style case lifecycle.
Starlette StaticFiles	Serves the single-page console from the same origin as the API.

The console (the `web/` side)

Vanilla HTML/CSS/JS	One self-contained <code>index.html</code> . No build step, no framework — it opens in any browser.
Leaflet.js	The interactive money-flow map; Esri dark-canvas tiles with a labels layer.
SVG overlay	The waypoint flow-lines are drawn in a screen-space SVG layer glued to the map (see Ch. 35).

Research track & tooling

PyTorch + PyTorch Geometric The graph-neural-network research in `graphx/` — temporal and heterogeneous GNN baselines explored as a future upgrade path (Ch. 41).

`pytest` The leakage test suite — the mechanical guarantee that features never see the future.

Make One-line reproducible pipeline: `make all`, `make test`, `make demo`, `make api`.

Microsoft Edge / Chromium Renders these very manuals from HTML to PDF (headless print-to-PDF + Paged.js).

9 • The repository, file by file

PART II • THE SYSTEM AT A GLANCE

A map of the whole tree, so you always know where a thing lives. The five packages mirror the five pipeline stages.

Path	What it does
<code>config.py</code>	Single source of truth for constants: the four Pune districts & centroids, horizons, graph caps, H3 resolutions, abstention & tier thresholds, corpus window, filesystem layout.
<code>contracts.py</code>	Every Pydantic model — the frozen data contract all modules code against.
<code>gen/params.py</code>	Every generator dial; <code>.sealed()</code> returns the shifted test distribution. The ML side must never import this.
<code>gen/atms.py</code>	~700 synthetic ATM terminals placed across the districts, tagged with H3 res-7/8 cells.
<code>gen/generator.py</code>	Builds the synthetic corpus: accounts, ~250k transactions, 600 complaints.
<code>graphx/trail.py</code>	Time-respecting BFS money-flow reconstruction with proportional taint attribution.
<code>graphx/features.py</code>	Nine graph features per node plus the logistic mule-node score.
<code>graphx/candidates.py</code>	Five-source candidate ATM generation (C1–C5).
<code>graphx/*_gnn_*.py</code>	The GNN research track — temporal & heterogeneous graph nets, sprint experiments (Ch. 41).
<code>ml/features.py</code>	The as-of feature store (<code>FeatureContext</code>) and every feature function.
<code>ml/panel.py</code>	Builds <code>(account, as_of)</code> training rows.
<code>ml/train.py</code>	Fits M1 (hazard), M2 (channel), M3 (location ranker).
<code>ml/predict.py</code>	Inference → a <code>Prediction</code> , including the hierarchical abstention logic.
<code>ml/evaluate.py</code>	Four evaluation splits, four baselines, reliability/coverage plots, lead-time.
<code>api/main.py</code>	FastAPI backend, decision layer, alerts, and the SQLite ticket endpoints.
<code>web/index.html</code>	The single-page officer console (~1,850 lines): Command Centre, Case view + map, Case Centre.
<code>tests/test_leakage.py</code>	Mechanical leakage guarantees (R1/R2/R3).
<code>demo.py</code>	Headless end-to-end replay — prints one case start to finish in the terminal.
<code>data/ • data_sealed/</code>	The training and sealed Parquet corpora (accounts, transactions, complaints, atms).
<code>models/ • reports/</code>	Trained model artefacts and the evaluation outputs.

10 · Why the data is synthetic

PART III · THE DATA FOUNDATION

Every row of data in this project is generated by our own code. That is not a shortcut — it is a legal necessity and, handled correctly, a scientific strength. This chapter explains both, because "your data is fake" is the first thing a sceptic will say, and there is a strong answer.

The legal reason

Real money-mule chains are drawn from live bank and complaint records. In India those are personal financial data protected under the **Digital Personal Data Protection Act, 2023**. They cannot lawfully be poured into an open prototype that students build and judges inspect. A synthetic world is the only responsible way to build and share this system.

The scientific reason it still counts

A synthetic dataset is only worthless if it is *easy* — if mules and honest customers look obviously different, a model "learns" nothing transferable. Our generator is built to be the opposite: **deliberately hard**. Mules and legitimate customers overlap heavily, so the model must learn subtle, genuine signal.

THE REALISM LEVERS (WHY THE LESSONS TRANSFER)

Overlapping mule and legitimate timing distributions · six legitimate archetypes that *look* like mules (a salaried worker withdrawing their salary, a small business making round-number deposits, a student with a fresh account) · partial observability (some inter-bank edges are simply unobserved, as in reality) · injected label noise · and a mule "ring" structure that shares ATMs. A model that scores well on this world is learning behaviour, not a watermark.

11 · The synthetic world

PART III · THE DATA FOUNDATION

The generator (`gen/generator.py`) builds a self-consistent little economy and then lets fraud happen inside it. The pieces:

Accounts (8,000)

Each account has a KYC-registered district and PIN (the only location the models may use), an opening branch, an opening date (so account age is a real feature), and a hidden *archetype* that drives its behaviour. About **6%** are mules. The archetype is never shown to the models — it lives only in a sealed internal table.

Archetypes — the overlap that makes it hard

Legitimate (94%)	Mule (6%)
SALARIED · SMB · STUDENT · SENIOR · FARMER — each with realistic, and deliberately mule-like, transaction rhythms.	MULE_FRESH (just recruited) · MULE_SEASONED (established) · MULE_RING (operated as a coordinated set).

Transactions (~250,000)

Ordinary economic activity — salaries, purchases, transfers, ATM withdrawals — plus the fraudulent flows. Every transaction has a timestamp, amount, channel, and an `observed` flag (some inter-bank edges are unobserved, modelling imperfect data sharing). Cash-out channels are ATM withdrawals and POS.

Complaints (600)

Each complaint names one victim and one seed transaction, records a filing time (with a realistic reporting delay), a disputed amount, and a fraud category. These 600 cases are what the console's queue is built from.

Rings (size 3)

Mules are grouped into small rings that share cash-out ATMs and coordinate. This is the single most useful fraud-specific signal, and it is why one of the five candidate-ATM sources (C4, Ch. 19) looks at what an account's ring co-members have done.

12 · The generator dials

PART III · THE DATA FOUNDATION

Every knob lives in `gen/params.py` as a frozen dataclass. Exposing them in one place makes the world reproducible and lets us build the "sealed" shift by simply changing a few numbers. The headline dials:

Dial	Default	Meaning
seed	26184	Master RNG seed (the SIH problem number) — full reproducibility.
n_accounts	8,000	Population size.
mule_frac	0.06	Share of accounts that are mules.
n_transactions_target	250,000	Approximate corpus size.
n_complaints	600	Number of fraud cases (the queue).
ring_size	3	Accounts per mule ring.
legit_archetype_mix	5 classes	SALARIED .34 · STUDENT .18 · FARMER .18 · SMB .16 · SENIOR .14.
mule_archetype_mix	3 classes	MULE_FRESH .40 · MULE_SEASONED .35 · MULE_RING .25.
mule_hotspot_mix	by district	Where mules cluster geographically (legit accounts are ~uniform).

RULE: THE ML SIDE MUST NEVER IMPORT GEN/PARAMS.PY

If any module under `ml/` or `graphx/` read a generator parameter, the model would be told the answer instead of learning it. The leakage test greps imports and fails the build if this boundary is crossed. The only legitimate readers are the generator itself and the evaluation harness (to build the sealed shift).

13 · ATMs, geography & H3 cells

PART III · THE DATA FOUNDATION

The pilot geography is the **Pune metropolitan area**, modelled as four districts, each with a real centroid and a covering cyber police station for alert routing.

District	Code	Centroid (lat, lon)	Station (alert target)
Pune City	PNC	18.5204, 73.8567	Pune City Cyber PS
Pimpri-Chinchwad	PCH	18.6279, 73.8009	Pimpri-Chinchwad Cyber PS
Hinjawadi	HJW	18.5913, 73.7389	Hinjawadi IT-Park PS
Hadapsar	HDP	18.5089, 73.9260	Hadapsar Cyber PS

ATMs

`gen/atms.py` scatters ~700 synthetic ATM terminals across these districts. Each terminal has an operator (bank), a PIN, a district, and — crucially — its H3 cells at two resolutions. The file also carries the exact OpenStreetMap Overpass query to swap in real `amenity=atm` nodes when the pilot goes live.

H3 — the spatial backbone

H3 is Uber's hexagonal grid index. A latitude/longitude collapses to a hexagon id at a chosen resolution. We use two:

- **Resolution 7 (~5 km²)** — the *operational cell*. This is the neighbourhood-sized area the location model reports when it is confident about the area but not the exact ATM.
- **Resolution 8 (~0.5 km²)** — the *fine tier*, a tighter grouping used inside the location features.

H3 is what makes the location *hierarchy* — district → cell → terminal — clean and computable: aggregating terminal probabilities up to their parent cell is just a group-by on the cell id.

14 · The sealed corpus

PART III · THE DATA FOUNDATION

The most common attack on any model trained on self-generated data is: *"your model just memorised your own generator."* The sealed corpus is the pre-built answer.

`Params.sealed()` returns a **deliberately different** distribution, and the model is evaluated on data built from it without ever training on it. The differences are structural, not cosmetic:

- a **different mule-hotspot mix** (fraud clusters in different districts);
- **slower forwarding delays** (the money moves at a different pace);
- **lower fraud prevalence** (rarer positives);
- **worse bank cooperation** (more unobserved edges);
- and a fraud typology — `smish_burst` — that **never appears in training at all**.

THE PAYOFF

Performance on the sealed corpus barely drops (ROC-AUC 0.913 → 0.879). A memoriser would collapse; a model that learned genuine behaviour degrades gracefully. That single before/after number is the strongest evidence in the whole project that the learning is real.

15 · The data contracts

PART III · THE DATA FOUNDATION

Every shape that crosses a module boundary is a Pydantic model in `contracts.py`. Freezing these first means the type checker finds every call site if a shape ever changes, and the API's JSON is validated automatically. The important contracts:

Contract	What it carries
Case	A complaint: <code>ack_no</code> , <code>filed_ts</code> , <code>victim_account</code> , <code>seed_txn_id</code> , <code>disputed_amount</code> , <code>fraud_category</code> , <code>reporting_delay_min</code> .
TrailNode / TrailEdge / Trail	The reconstructed money flow: nodes (with <code>taint</code> , <code>is_victim</code> , <code>is_leaf</code> , KYC lat/lon/district/pin) and edges (<code>amount</code> , <code>tainted_amount</code> , <code>channel</code> , <code>hop</code> , <code>atm_id</code> if a cash-out).
HazardPoint / CashoutPrediction	M1 output: cumulative cash-out probability at 10/20/30/45/60 min, the action window, and a severity label.
ChannelPrediction	M2 output: the four probabilities ATM / POS / onward_transfer / dormant.
LocationCandidate / CellPrediction / DistrictPrediction / LocationPrediction	M3 output: ranked terminals, cells and districts, the granularity actually emitted, and the abstention fields.
Recommendation	The decision-layer verdict: tier, action (FREEZE/DISPATCH/MONITOR), a human-readable reason, freeze target & deadline, target station.
Prediction	The top-level object <code>/predict</code> returns: cashout + channel + location + recommendation for one account.
Alert / QueueItem	The dispatched notification, and one row of the triage queue (with golden-hour-remaining and an imminence sort key).

R6 — NAIVE IST TIMESTAMPS

All datetimes are stored as naive objects understood to be IST. The `+05:30` offset appears in exactly one place: JSON serialisation. This keeps the pipeline from ever juggling timezone-aware objects, which is a classic source of subtle bugs.

16 · The as-of principle

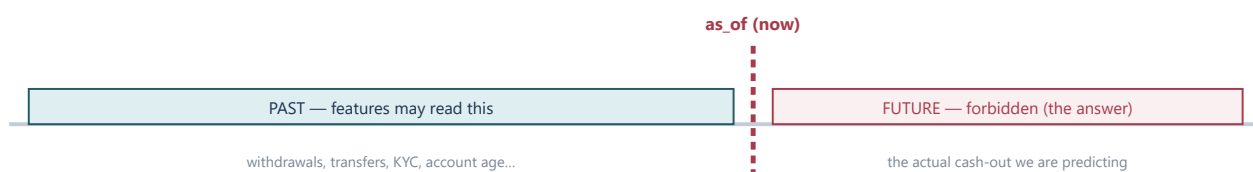
PART IV · TRACING THE MONEY

This is the single most important idea in the entire project. Everything that makes the results credible flows from it. If you take one concept away from this manual, take this one.

A forecast is a statement about the future. If, while computing it, you accidentally use any information that only exists *after* the moment of the forecast, you have cheated — you have "peeked at the answer." Your accuracy will look spectacular in testing and collapse in the real world. This mistake is called **data leakage**, and it is the number-one reason machine-learning prototypes fail when deployed.

R1 — THE AS-OF RULE

Every feature in this system is a function `f(account_id, as_of, ctx)` and may read **only** data with `ts < as_of`. The graph is rebuilt fresh at every `as_of`. There is no precomputed whole-corpus graph — that would be the first and worst leak.



The as-of line. Only data to the left of it may inform a prediction.

Enforcing this is not a matter of good intentions — humans forget. It is enforced *mechanically* (Ch. 27): the leakage test rebuilds a physically truncated corpus (all future rows deleted) and asserts that features computed on it are bit-for-bit identical to features computed with the as-of filter on the full corpus. If a feature ever peeks, the two disagree and the build fails.

17 · Money-flow reconstruction

PART IV · TRACING THE MONEY

`graphx/trail.py` turns "one victim, one seed transaction, one moment" into the graph of where the money went. It is a breadth-first walk outward from the seed, and it obeys the as-of rule at every step.

The algorithm

1. Start at the seed edge: victim → first mule, fully tainted.
2. From each account, follow only outgoing edges whose timestamp lies strictly between the money's arrival and `as_of` (delegated to the feature store so the filter is applied identically everywhere).
3. Ignore dust: edges worth less than **5%** of the seed amount (`MIN_EDGE_FRACTION`) are dropped.
4. Attribute taint **proportionally** through each split: if an account received ₹100 of stolen money and forwards ₹60 of a ₹300 total outflow, the tainted portion of that edge is scaled accordingly, capped at the edge amount.
5. Stop at depth **4** (`MAX_HOPS`) or when a cash-out ends the chain.

What comes out

A `Trail` of nodes and edges. A node is flagged `is_leaf` when it received more than it forwarded — i.e. it is **currently holding stolen funds**. Those leaves are the forecast targets: the accounts a freeze could still save. Each node also carries its KYC centroid (lat/lon/district/pin) so the console can plot the flow, and its total tainted amount so the decision layer can size the risk.

WHY REBUILD EVERY TIME?

Because the trail as-of 10:05 is a different graph from the trail as-of 10:25 — more money has moved. A precomputed graph would silently contain future edges. Rebuilding is cheap (the BFS is capped and pruned) and it is the only way to keep R1 honest.

18 · Graph features & the mule score

PART IV · TRACING THE MONEY

`graphx/features.py` computes nine numbers per node that capture *where in the fraud structure* an account sits — signal you cannot get from an account's own transactions alone. All nine are as-of correct.

#	Graph feature	Intuition
1	hop_from_victim	How many layers deep in the laundering chain.
2	in_deg_1h	Incoming edges in the last hour — a mule receiving from many.
3	out_deg_1h	Outgoing edges in the last hour — fast fan-out.
4	dispersion_ratio	How widely the money is being spread onward.
5	flow_concentration	How concentrated inflows are (one big source vs many).
6	ring_size	Size of the account's co-operating ring.
7	ring_prior_cashout_rate	How often this ring's members have cashed out before.
8	n_tainted_predecessors	How many upstream accounts carry stolen money.
9	mule_score	A small logistic regression over features 1–8 — a single "how mule-like" number.

THE MULE SCORE IS A FEATURE, NOT A HEADLINE

Feature 9 is not a separate model competing with M1. It is one column that M1 consumes. Keeping it as a feature (rather than a standalone classifier) means its uncertainty is folded into the hazard model honestly, and it stays as-of correct like everything else.

19 · Candidate ATM generation

PART IV · TRACING THE MONEY

The location model (M3) does not score all 700 ATMs — that would be slow and mostly noise. Instead, `graphx/candidates.py` first proposes a *shortlist* of plausible ATMs for an account, and M3 only ranks those. The shortlist is the union of five sources, in priority order, capped at 50 (`CANDIDATE_CAP`):

Source	~n	Where the candidates come from
C1	15	Every ATM this account itself used in the last ~90–120 days (its own habit).
C2	12	ATMs within 4 km of the account's historical withdrawal centroid.
C3	10	ATMs within 3 km of its KYC PIN centroid (falls back to the opening branch).
C4	10	The differentiator: top ATMs used by the account's <i>ring co-members</i> in 180 days.
C5	8	The district's top fraud-withdrawal ATMs — train period only.

CANDIDATE RECALL IS THE CEILING

If the real ATM is not in the shortlist, no ranker can ever find it. So *candidate recall@50* — the fraction of cases where the true ATM is somewhere in the shortlist — is a hard ceiling on location accuracy. Ours is **0.82**, and source C4 (the ring signal) is the single biggest contributor. This is why modelling the ring mattered.

20 · The as-of feature store

PART V · THE FORECASTING ENGINE

The `FeatureContext` in `ml/features.py` is the workhorse that makes as-of feature computation both correct and fast. Understanding it is understanding how the leakage rule is kept without making the system unusably slow.

On startup, the context loads all transactions once and pre-sorts them per account by timestamp. Then any as-of query — "this account's withdrawals before 10:05" — is answered with a binary search (`numpy.searchsorted`) that finds the cut-point in a sorted array in microseconds, rather than filtering 250,000 rows every time. The same cut is applied through one code path (`_before`), so every feature, every model, and the leakage test all slice time *identically*. That single shared cut is what makes the bit-for-bit leakage guarantee possible.

The context also exposes the small priors the models legitimately use: an ATM's historical fraud rate, a district's hotspot prior, an account's KYC info and ring membership, and helpers like the account's withdrawal centroid and radius of gyration.

21 · Every feature, grouped

PART V · THE FORECASTING ENGINE

The complete feature vocabulary, organised by what each group captures. Every one is computed only from `ts` < `as_of`.

Velocity & money-movement

`taint_amt`, `mins_since_taint`, `pct_forwarded_5m/10m/30m`, `n_tx_5m/10m/30m`, `mins_since_last_tx`, `inflow_outflow_ratio`, `largest_debit_ratio`, `round_amount_flag`. These answer: how much stolen money is here, how fast is it moving, and does the pattern look like a pass-through?

Account history

`account_age_days`, `mean_txn_amt_90d`, `amt_zscore_vs_history`, `dormancy_break_days`, `n_new_counterparties_24h`. A freshly opened account suddenly transacting large round sums to new parties is a classic mule tell.

Graph position (from Ch. 18)

`hop_from_victim`, `in_deg_1h`, `out_deg_1h`, `dispersion_ratio`, `flow_concentration`, `ring_size`, `ring_prior_cashout_rate`, `n_tainted_predecessors`, `mule_score`.

Spatial (location model)

`prior_atm_count_90d`, `n_distinct_atms_90d`, `radius_of gyration_km`, `dist_kyc_to_centroid_km`. An account's *habitual* withdrawal geography — never its live location.

Temporal

`modal_withdrawal_hour`, `hour_sin`, `hour_cos`, `dow`, and `mins_complaint_to_now`. Time-of-day is encoded as sine/cosine so "23:00" and "01:00" are correctly close.

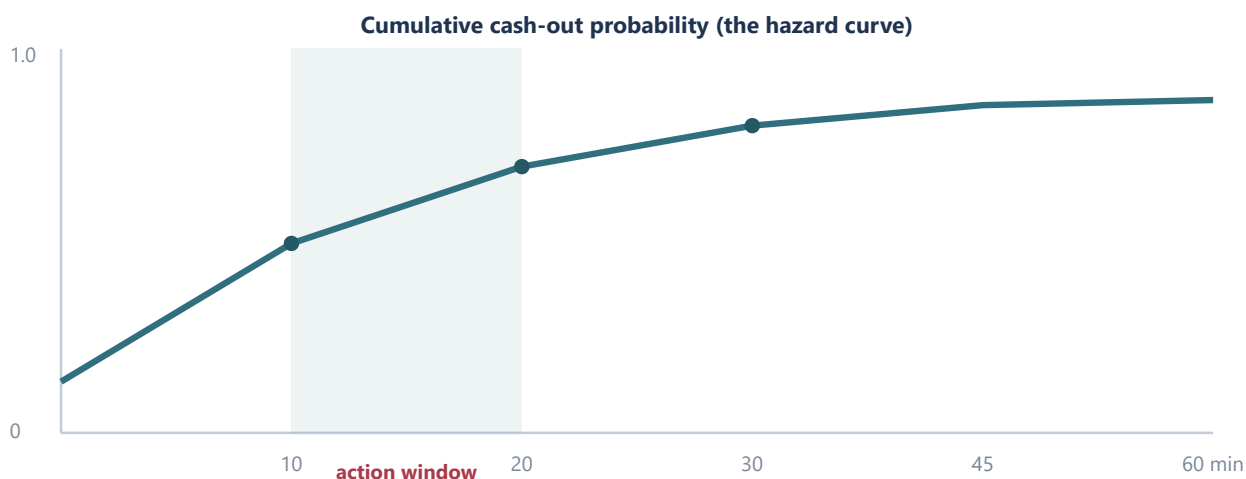
THE FOUR CHANNEL CLASSES

M2 predicts over exactly four outcomes: `ATM`, `POS`, `onward_transfer`, `dormant`. The last two matter operationally: "onward_transfer" tells police *not* to rush an ATM because the money is about to move again.

22 · M1 — the hazard curve

PART V · THE FORECASTING ENGINE

M1 answers "**will this account be cashed out, and when?**" It is a discrete-time hazard model: a gradient-boosted classifier scored at each horizon in {10, 20, 30, 45, 60} minutes, with the horizon itself as a feature. The five scores are stitched into a *cumulative* curve (forced to be non-decreasing — the chance of cash-out within an hour can't be lower than the chance within ten minutes).



The steepest-rise segment of the hazard curve is read off as the "act by" window.

From the curve, M1 reads a **window** (the steepest-rise interval — when the risk is climbing fastest) and a **severity** label: *imminent* ($\geq 60\%$ within 10 min), *soon* ($\geq 30\%$), *later* ($\geq 10\%$), or *unlikely*. The 10-minute probability, `p_10m`, is the headline number the decision layer keys on.

23 · M2 — the channel

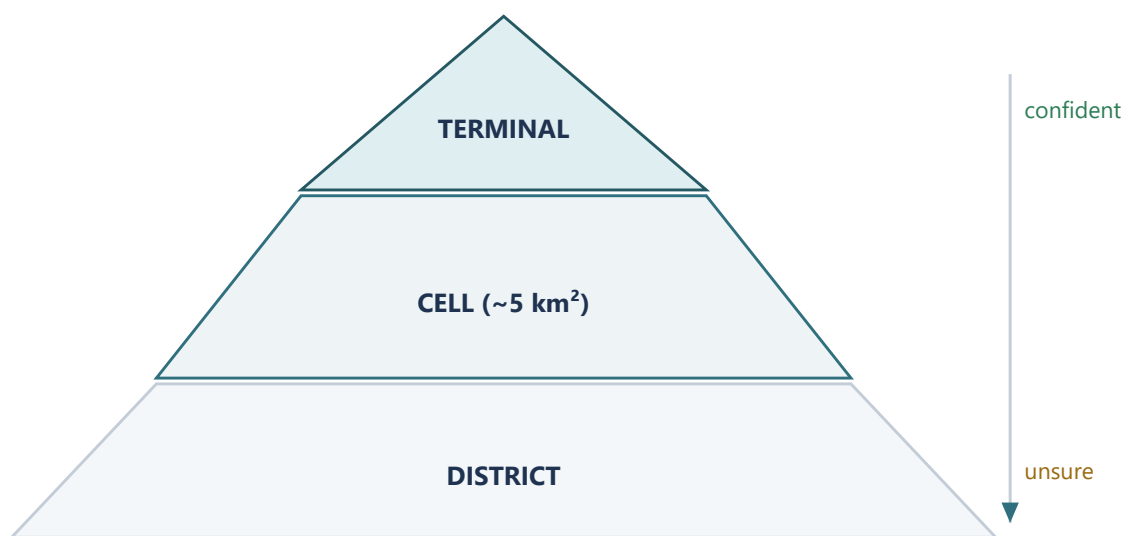
PART V · THE FORECASTING ENGINE

M2 answers "**how will the money leave?**" — a four-class classifier over ATM, POS, onward_transfer, and dormant. It exists to stop a wasteful mistake: sending a patrol to an ATM when the money is actually about to be forwarded again (onward_transfer) or is going nowhere right now (dormant). It sharpens the action, not just the probability.

24 · M3 — location & abstention

PART V · THE FORECASTING ENGINE

M3 answers "**where?**" — and it is the model the project is proudest of, because of how it handles *not knowing*. It ranks the candidate ATMs from Chapter 19, but then it does something most models refuse to do: it decides *at what level of precision it is willing to commit*.



Hierarchical abstention: the model emits the finest level it can defend, and no finer.

R5 — HIERARCHICAL ABSTENTION

M3 computes the terminal probability distribution and its entropy, then applies a two-gate rule:

- if the top terminal's probability ≥ 0.22 and entropy ≤ 2.60 nats $\rightarrow$ emit a **terminal** (the top 5 ATMs);
- else if the top cell's probability ≥ 0.30 $\rightarrow$ emit a **cell**, and record that it *abstained at terminal*;
- else $\rightarrow$ emit only the **district**, abstaining at cell.

The output always carries `abstained_at` and a plain-English `abstain_reason`, so the officer sees exactly why the system would only name a neighbourhood.

This is the "the system knows what it doesn't know" property. On a flat distribution, naming a single ATM would be a coin-flip dressed up as certainty; abstaining to the cell is both more honest and more *useful* (a patrol can watch a neighbourhood; it cannot stake out an ATM it was wrongly sent to). Judges consistently reward this because it is the opposite of the usual demo temptation to over-claim.

25 · Training & inference

PART V · THE FORECASTING ENGINE

Training (`ml/train.py` , `ml/panel.py`)

`ml/panel.py` builds the training table: for many `(account, as_of)` pairs it computes the full as-of feature vector and attaches the true outcome. `ml/train.py` then fits the three models and the mule-score logistic, and serialises them to `models/*.joblib` together with their exact feature lists (so inference uses the identical columns in the identical order).

Inference (`ml/predict.py`)

Given an account and an `as_of` , inference builds the feature vector once, runs M1 across the horizons to form the hazard curve, runs M2 for the channel, generates candidates and runs M3 with the abstention gates, and returns a single `Prediction` . This is the object the API hands to the decision layer and the console.

26 · The six non-negotiable rules

PART VI · HONESTY & EVALUATION

This is the part to read twice. Six rules keep the science honest, and — crucially — each is enforced by machinery, not by hoping the team remembers. When a judge probes "how do I know you didn't cheat?", these six answers are the reason the project holds up.

Rule	What it forbids	How it is enforced
R1 as-of	Using any data from after the prediction moment.	Every feature is <code>f(id, as_of, ctx)</code> ; the leakage test rebuilds a truncated corpus and asserts identical features.
R2 label prefix	A label reaching a model.	<code>drop_leaky_columns</code> strips every <code>label_*</code> column on sight.
R3 no live location	Using a suspect's real-time GPS/phone location.	Only KYC centroid, opening branch, and historical withdrawal footprint are exposed; true home coordinates live in a sealed internal table and are never shipped.
R4 honest metrics	Suspiciously high scores that signal a leak.	Target ROC-AUC 0.86–0.92; a single-feature audit flags anything scoring >0.85 alone. We <i>widened the generator</i> until 0.913 was true, never tuned upward.
R5 abstention	Naming a precise location on a flat distribution.	The district→cell→terminal gates in M3, with <code>abstained_at</code> / <code>abstain_reason</code> .
R6 naive IST	Timezone bugs from mixed tz handling.	All datetimes naive IST internally; <code>+05:30</code> appended only at JSON serialisation.

WHY R4 SOUNDS BACKWARDS — AND IS THE POINT

Most teams chase the highest possible accuracy. We *capped* our ambition at 86–92% and treat any single feature that alone scores above 0.85 as a bug to hunt, because in this domain a too-good number almost always means a leak. Deliberately *lowering* the ceiling to keep it honest is exactly the discipline a real deployment needs.

27 · The leakage test suite

PART VI · HONESTY & EVALUATION

`tests/test_leakage.py` is the mechanical conscience of the project. It runs under pytest and turns the rules above into failing builds. Its checks:

- **As-of identity (R1).** Build a corpus physically truncated at `as_of` (all later rows deleted). Compute features on it, and on the full corpus with the as-of filter. Assert they are *bit-for-bit* identical. Any feature that peeks makes them differ.
- **Label containment (R2).** Assert no `label_*` column survives into a model's input.
- **Location containment (R3).** Assert the generator's ground-truth home coordinates never appear in any shipped table or feature.
- **Import boundary.** Grep the `ml/` and `graphx/` trees and fail if any of them import `gen/params.py` (which would mean reading the answer key).

THE ONE-LINE PITCH FOR THIS SUITE

"We don't ask you to trust that we didn't cheat — we wrote tests that fail the build if we did, and they run on every change."

28 · Evaluation: splits & baselines

PART VI · HONESTY & EVALUATION

`ml/evaluate.py` scores the models four different ways and against four baselines, then writes plots to `reports/`. The four splits each test a different kind of generalisation:

Split	Tests	M1 ROC-AUC	Terminal Top-1	Cell Top-1
A chronological	future cases after a time cut	0.913	0.207	0.69
B unseen account	accounts never seen in training	0.916	0.267	0.73
C unseen terminal	ATMs never seen in training	—	0.146	0.68
D sealed (shifted)	a different distribution entirely	0.879	0.210	0.67

The four baselines answer "why not something simpler?":

rule engine	ROC-AUC 0.500 — a coin flip. This is the direct answer to "why not just use hand-written rules?"
logistic regression	0.840 — decent, but well below M1's 0.913.
nearest-KYC ATM	Top-1 0.057 for location — barely better than random.
most-frequent ATM	Top-1 0.198 — a strong-ish habit baseline that M3 (0.207) still beats while also abstaining honestly.

29 · The numbers & how to defend them

PART VI · HONESTY & EVALUATION

The figures to memorise, each with the sentence that defends it.

Number	What it is	How to defend it
0.913	M1 ROC-AUC (chronological)	"Honest, not inflated — we capped our target at 0.92 and audit any single feature over 0.85."
0.879	M1 on the sealed corpus	"Barely drops on a distribution we never trained on — it generalises, it didn't memorise."
0.82	candidate recall@50	"The real ATM is in our shortlist 82% of the time — the hard ceiling on location, driven by the ring signal."
~0.7 km	median location error when we name a place	"Neighbourhood-accurate when we commit — and we abstain when we can't."
50% vs 91%	rules baseline vs our model	"That gap is the entire answer to 'why not just use rules?'"
~12 min	median lead time when we can act	"When action is possible, we buy ~12 minutes — enough to freeze."
~66%	cases already cashed out before filing	"The honest reality, and the case for faster reporting and auto-freeze."

THE NORTH STAR

Lead time is the metric that actually matters operationally, and we report it warts and all. In the two-thirds of cases where the money is already gone, no model can help — which is precisely why the project argues for structural fixes (bank-side auto-alerts, proactive freezing) rather than pretending the AI alone solves it.

30 · The plain-rules decision layer

PART VII · DECISION & ALERTS

The models produce probabilities. Turning a probability into an *action* is done by plain, readable rules — deliberately not another model — because an officer must be able to read exactly why an alert fired, and a court must be able to audit it.

Tier	Rule	Action
RED	$p_{10m} \geq 0.60$ AND tainted amount $\geq ₹5,000$	FREEZE now + alert the station.
AMBER	$0.30 \leq p_{10m} < 0.60$	Watch / email a brief.
GREY	below AMBER	Keep tracing.

The amount floor is set realistically: after fan-out the money is split across mules, so a single account rarely holds the whole disputed sum — but a 60%-imminent cash-out of even ~₹15,000 is worth a freeze, because freezing is cheap and letting it clear is not. Every recommendation carries a plain-English *reason string* — the rule trace — so nothing about the verdict is mysterious.

31 · Alert routing & dedup

PART VII · DECISION & ALERTS

When a case fires, an alert is built and logged (SMS / email / webhook payloads). Routing uses the jurisdiction table in `config.py`: each district maps to a covering cyber police station and its SHO contact, so a Hadapsar cash-out alerts the Hadapsar Cyber PS. To avoid spamming, alerts are de-duplicated: at most one per account per **15 minutes** (`ALERT_DEDUP_MINUTES`).

SIMULATED, AND HONEST ABOUT IT

The system *builds and logs* the notification payloads; it does not actually send them. That is the correct posture for a prototype and is stated plainly — the plumbing is real, the delivery is stubbed.

32 · The API, endpoint by endpoint

PART VII · DECISION & ALERTS

The FastAPI backend (`api/main.py`). Interactive docs are served at `/docs` .

Method & path	Purpose
GET <code>/queue</code>	The triage queue — active complaints sorted by imminence, with golden-hour-remaining.
GET <code>/case/{ack_no}</code>	The full case record for one complaint.
GET <code>/trail/{ack_no}</code>	The reconstructed money-flow trail (nodes + edges) as-of now.
GET <code>/predict/{ack_no}</code>	The full forecast for the case's top account: cashout + channel + location + recommendation.
GET <code>/hotspots</code>	Predicted risk per H3 cell across the complaint window + ATM markers + district centroids. Supports date/amount/channel filters.
POST <code>/alerts/dispatch</code>	Build, log and "send" an alert for a case.
GET <code>/alerts</code>	The alert delivery log.
GET <code>/explain/{ack_no}</code>	A human-readable explanation of the forecast and the rule trace.
GET <code>/replay/state</code>	The state of a case replayed in fast motion (drives the cinematic replay).
GET <code>/tickets</code> · <code>/tickets/summary</code> · <code>/tickets.csv</code>	The Case Centre ticket list, summary counts, and CSV export.
GET <code>/tickets/{ack_no}</code>	One ticket's full detail and audit trail.
PATCH <code>/tickets/{ack_no}</code>	Assign or change a ticket's status.
POST <code>/tickets/{ack_no}/note</code> · <code>/resolve</code>	Add an investigation note; resolve/close a ticket with an outcome.

33 · Design philosophy

PART VIII · THE CONSOLE

The console is one HTML file with no build step. Its job is to make a complex forecast legible to an officer under time pressure, and to make the money move visibly so the story is obvious. The visual language is a calm, mature command-console — dark cartography, restrained accents, and a single clear "what do I do" per case.

An earlier iteration leaned neon; the current theme is deliberately toned down to a muted steel-teal palette with soft glows, so it reads as a serious operations console rather than an arcade. The three accent colours carry fixed meaning everywhere: teal = information, amber = watch, red = act/urgent.

34 · The three views

PART VIII · THE CONSOLE

Command Centre

The situational overview: cases tracked, freeze-now count, money at risk, average window left, and a metro-wide risk map. It answers "what is the state of play right now?" at a glance. A collapsible sidebar lists active cases, most urgent first, each with its risk tier and golden-hour clock.

Case view

The heart of the product for a single case. It shows the live money-flow map (Ch. 35), the ranked target-ATM menu, the hazard/channel/location forecast, the freeze recommendation, and a replay control. Clicking a case loads it here.

Case Centre

The case-management workspace (Ch. 37): the ticket queue, filters, per-ticket detail, and the resolution workflow with an audit trail.

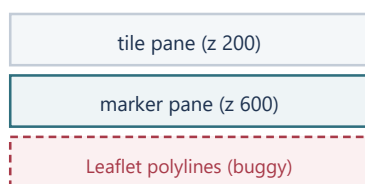
35 · The live money-flow map

PART VIII · THE CONSOLE

The map is Leaflet with a dark Esri canvas base and a labels layer so the officer can read the actual neighbourhoods. Nodes are drawn as clean circular icons: the victim, the mule accounts (the holding mule highlighted), and the ranked candidate ATMs. Money flows are drawn as animated "waypoint" lines: gold for observed flow (victim → mules), red for the predicted routes (holding mule → each candidate ATM, #1 the primary).

The one non-obvious engineering detail worth knowing

The connecting lines are **not** drawn as ordinary Leaflet polylines. Leaflet paints markers above polylines and, under the large render padding this map uses, its SVG layer can mis-position paths by the padding offset — which made the lines vanish under the icons or off-screen. The fix: the routes are drawn in a dedicated **screen-space SVG overlay** that sits above the whole map and is redrawn every frame from `latLngToContainerPoint` — the map's own pixel coordinates. This glues the lines pixel-perfectly to the icons regardless of zoom, pan, or resize. Each route is also trimmed to stop at the *edge* of each circle (by the node's radius) so the icons and their rank numbers stay clean.



Leaflet paints markers above lines →
clustered lines vanish under the dots.



Why the waypoint lines live in a screen-space overlay, not in Leaflet's own SVG layer.

36 · The cinematic replay

PART VIII · THE CONSOLE

The single best demo moment. Pressing *Replay simulation* re-runs a real case in fast motion inside a cinematic frame: a letterboxed view with a running timecode and act labels ("MONEY ENTERS", "TRACING THE FLOW", "CONVERGING ON CASH-OUT", "CASH-OUT WINDOW — FREEZE"). The camera is held fixed on the whole case so the money reveals systematically; the observed gold flow traces first, and the predicted red target routes reveal only near the climax — mirroring how the real forecast sharpens as the golden hour closes. It turns an abstract probability into a visible, tense story an audience can follow.

37 · The SOAR ticket lifecycle

PART VIII · THE CONSOLE

The Case Centre turns forecasts into managed casework, in the style of a security operations (SOAR) console. Each case becomes a ticket with a priority (P1/P2/P3 derived from the RED/AMBER/GREY tier), a status, an assignee, investigation notes, and a resolution outcome — all backed by a small SQLite store (WAL mode, seeded once). Officers can filter the queue, open a ticket to see its full forecast and audit trail, assign it, add notes, and resolve it (e.g. *frozen in time, money recovered*). A printable incident report and a CSV export round it out. This is what makes the prototype feel like an operational tool rather than a single-shot demo.

38 · Setup from zero

PART IX · RUNNING & OPERATING

From a clean checkout to a running console. Everything runs locally; no cloud, no keys.

1 · Environment

```
# from the sih26184/ folder, with Python 3.12
python -m venv .venv
.venv\Scripts\activate
pip install pandas numpy scikit-learn xgboost networkx \
    fastapi uvicorn pydantic pyarrow h3 matplotlib pytest
```

2 · Build the world & train (once)

```
python -m gen.atms          # ATM table      -> data/atms.parquet
python -m gen.generator     # training corpus -> data/
python -m gen.generator --sealed # sealed test set -> data_sealed/
python -m ml.train          # fit M1, M2, M3  -> models/
python -m ml.evaluate       # 4 splits + plots -> reports/
```

3 · Verify honesty

```
python -m pytest tests/ -q      # the leakage suite must pass
```

4 · Run it

```
python demo.py                # headless end-to-end in the terminal
python -m uvicorn api.main:app --port 8000 # then open http://localhost:8000
```

Or, with the Makefile: `make all && make test && make demo && make api`.

39 · The live demo script

PART IX · RUNNING & OPERATING

A three-minute run that lands every key point in order.

1. **Open the console.** "This is the officer's screen — the Command Centre. Left is the queue of live complaints, sorted by urgency."
2. **Click the top RED case.** "The system traced the money to this account, which is holding ₹X of stolen funds."
3. **Point at the hazard curve.** "It forecasts a high chance of cash-out in the next ten minutes, and reads off the action window."
4. **Point at the map & the location.** "Most likely in this neighbourhood — and notice it *abstains* from naming a single ATM when it isn't certain. Honest AI."
5. **Point at the recommendation.** "So the action is FREEZE, and an alert routes to this specific police station."
6. **Hit Replay.** "Here's the same case in fast motion — watch the money move and the risk rise, then the cash-out window open."
7. **Close on the number.** "91% accuracy versus 50% for a rules engine — and it tells us when it doesn't know."

STAGE SAFETY NET

If the browser dies on stage, run `python demo.py` — it prints the whole story to the terminal, so the demo never depends on the UI staying up.

40 · Troubleshooting

PART IX · RUNNING & OPERATING

Symptom	Cause & fix
Server won't start: "No module named uvicorn"	Wrong interpreter. Activate the venv (<code>.venv\Scripts\activate</code>) or call <code>.venv\Scripts\python.exe -m uvicorn ...</code> .
Empty map / "select a case"	The backend is still initialising (loading data & models into the FeatureContext). Wait a few seconds and reload.
Model files missing	Run the build steps: <code>gen.atms → gen.generator → ml.train</code> .
A district KeyError at inference	Stale data after a geography change; the location code falls back to the district's terminal-mean centroid, but rebuild the corpus to be safe.
Map tiles blank	Esri tiles occasionally 404 for a z/x/y; an OpenStreetMap fallback kicks in after a few misses.

41 • The GNN research track

PART X • RESEARCH & ROADMAP

The shipping models (M1–M3) are gradient-boosted trees — reliable, fast, and easy to defend. Alongside them, the `graphx/` tree contains a substantial research track exploring graph neural networks as a future upgrade, built on PyTorch and PyTorch Geometric.

This work is organised as a series of "sprints" and baselines:

- **Graph & hetero baselines** — plain and heterogeneous GNN classifiers over the account graph, to test whether learned message-passing beats hand-built graph features.
- **Temporal GNNs (TGNN)** — models that respect the as-of ordering and learn from the *sequence* of movements, not a static snapshot.
- **Candidate-ceiling & ATM-ranker studies** — measuring how high location accuracy could go if candidate recall improved, and pairwise-ranking alternatives to the current softmax ranker.
- **Explainability & final validation** — sprint 11/12 work on explaining GNN outputs and validating them under the same leakage discipline as the shipping models.

WHY TREES SHIP AND GNNS WAIT

For a time-critical, auditable deployment, a boosted-tree model that a court can inspect and that trains in seconds beats a heavier GNN that is harder to explain. The GNN track is the documented path to *more* accuracy once the operational basics are proven — not a dependency of the current system.

42 · Known limitations

PART X · RESEARCH & ROADMAP

Stated plainly, because credibility comes from naming your own weaknesses first.

- **ATM coordinates are synthetic**, scattered around real district centroids; `gen/atms.py` holds the exact Overpass query to swap in real ATMs.
- **All training data is synthetic by design** (DPDP Act). The generator is deliberately hard so the modelling lessons transfer, but it is not real mule data.
- **Not a licensed forecast of any real person or account**. Outputs are decision-support for authorised investigators, not evidence.
- The location ranker is the *simple* binary-classifier version (softmax within group); a pairwise ranker is a documented next step.
- **Alerts are simulated** — payloads are built and logged, not delivered.

43 · Roadmap to production

PART X · RESEARCH & ROADMAP

1. **Real geography.** Swap synthetic ATMs for OpenStreetMap `amenity=atm` nodes (query already in the code).
2. **Bank integration.** Wire the freeze recommendation to a real account-freeze API, and the alerts to real SMS/email/webhook gateways.
3. **Faster reporting.** The biggest single win is upstream: bank-side auto-alerts that shrink the ~110-minute reporting delay, widening the window the model can act in.
4. **Model upgrades.** Promote the temporal GNN track if it beats trees under the same leakage discipline; add the pairwise location ranker.
5. **Hardening.** Authentication, per-station access control, and audit logging on every action for evidentiary use.

Appendix · Command & file reference

APPENDIX

Command reference

<code>make all</code>	Build data + train + evaluate, end to end.
<code>make test</code>	Run the leakage suite.
<code>make demo</code>	Headless end-to-end replay in the terminal.
<code>make api</code>	Start the console backend on :8000.
<code>python -m gen.atms</code>	Build the ATM table.
<code>python -m gen.generator [--sealed]</code>	Build the training (or sealed) corpus.
<code>python -m ml.train</code>	Fit M1/M2/M3 + mule score.
<code>python -m ml.evaluate</code>	Four splits, four baselines, plots.
<code>python -m pytest tests/ -q</code>	Leakage guarantees.
<code>uvicorn api.main:app --port 8000</code>	Run the API + console.

The six rules, one line each

R1	as-of: features read only <code>ts < as_of</code> .
R2	label prefix: every <code>label_*</code> column is dropped.
R3	no live location: only KYC + historical footprint.
R4	honest metrics: target 0.86–0.92; audit anything >0.85 alone.
R5	abstention: district → cell → terminal, only as far as defensible.
R6	naive IST: <code>+05:30</code> appended only at serialisation.

SIH26184 · Predictive Analytics for Cybercrime Cash-out Forecasting · The Complete Manual · Version 1.0. All data synthetic by design. Decision-support for authorised investigators, not evidence. Generated with Claude Code.